

Phát triển ứng dụng web

ReactJS
(part 1)

Nội dung

- ☐ ReactJS – Mở đầu
- ☐ ReactJS – Components
- ☐ ReactJS – Props
- ☐ ReactJS - TailwindCSS
- ☐ ReactJS – State

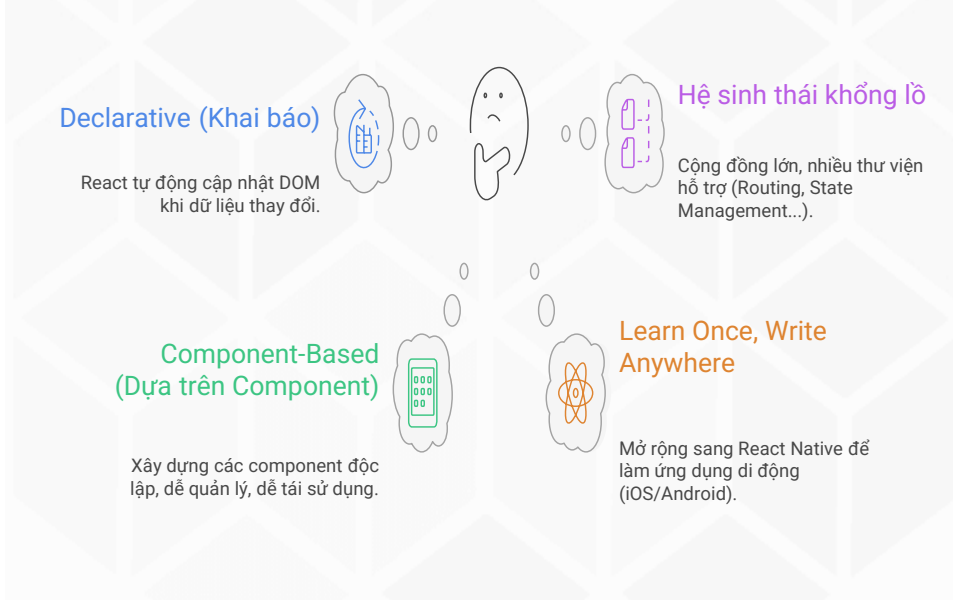
Nội dung

- ☐ **ReactJS – Mở đầu**
- ☐ ReactJS – Components
- ☐ ReactJS – Props
- ☐ ReactJS - TailwindCSS
- ☐ ReactJS – State

ReactJS là gì?

- ☐ Là một thư viện JavaScript (**library**), không phải là framework.
- ☐ Được phát triển và bảo trì bởi Facebook (Meta).
- ☐ Dùng để xây dựng giao diện người dùng (User Interfaces - UI).
- ☐ Core ý tưởng: Xây dựng UI từ các **khối nhỏ, độc lập**, có thể **tái sử dụng** gọi là **Components**.

Tại sao chọn React?



VITE – Công cụ tối ưu cho React?

❑ VITE là gì?

- ❑ Là một công cụ build (build tool) và development server thế hệ mới.
- ❑ Siêu nhanh (Blazing Fast)!!!

❑ VITE vs. Create React App (CRA)

Tính năng	Create React App (CRA)	VITE
Dev Server	Dựa trên Webpack Dev Server.	Dựa trên Native ES Modules (ESM) .
Tốc độ khởi động	Chậm . Phải bundle toàn bộ ứng dụng trước.	Cực nhanh . Khởi động server gần như tức thì.
Hot Module Replacement (HMR)	Chậm.	Cực nhanh . Chỉ cập nhật module bị thay đổi.
Cấu hình	Khó (cần "eject" hoặc craco).	Dễ dàng (dùng vite.config.js).

Thiết lập môi trường

☐ Node.js:

- ☐ Phiên bản **18.x** trở lên.
- ☐ Kiểm tra phiên bản: **node -v**
- ☐ Nơi tải: <https://nodejs.org/> (Nên chọn bản LTS)

☐ Trình soạn thảo code (Code Editor):

- ☐ Khuyến nghị: **Visual Studio Code (VS Code)**.
- ☐ Các extensions: **ES7+ React/Redux/Native snippets, Prettier - Code formatter**.

Bắt đầu tạo project

☐ Sử dụng lệnh: **npm create vite@latest**

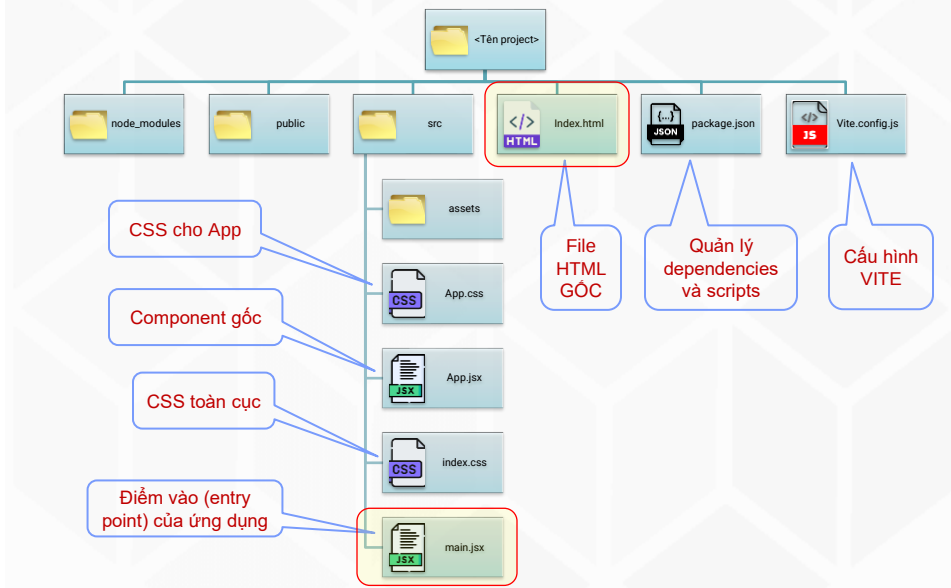
☐ Các bước lựa chọn:

- ☐ Project name
- ☐ Select a Framework: chọn React
- ☐ Select a variant: lựa chọn JavaScript hoặc JavaScript + SWC

☐ Chạy project vừa tạo

- ☐ Di chuyển vào folder project (nếu cần)
- ☐ Thực hiện cài đặt các gói modules: **npm i**
- ☐ Chạy server ở chế độ dev: **npm run dev**

Cấu trúc thư mục VITE tạo cho ReactJS



Nội dung

- ☐ *ReactJS – Mở đầu*
- ☐ **ReactJS – Components**
- ☐ ReactJS – Props
- ☐ ReactJS - TailwindCSS
- ☐ ReactJS – State

Components là gì?

- ❑ Là các khối xây dựng (building blocks) cơ bản trong React.
- ❑ Là các hàm JavaScript (hoặc class) trả về các phần tử React (thường là JSX).
- ❑ Giúp chia nhỏ UI thành các phần độc lập, có thể tái sử dụng.

Functional Components

- ❑ Đây là cách viết component hiện đại và phổ biến nhất trong React.
- ❑ Là một hàm JavaScript **bình thường**.
- ❑ Tên hàm **bắt buộc** phải viết hoa chữ cái đầu (PascalCase).
- ❑ Hàm này phải **return** về JSX.

```
function Welcome() {  
  return <h1>Hello, world!</h1>;  
}
```

```
const Welcome = () => {  
  return <h1>Hello, world!</h1>;  
};
```

Sử dụng Component

```
import Welcome from "../components/Welcome";  
function App() {  
  return (  
    <div>  
      <h1>Starting with Components</h1>  
      <Welcome />  
    </div>  
  );  
}  
export default App;
```

```
<div id="root">  
  <div>  
    <h1>Starting with Components</h1>  
    <h1>Hello, world!</h1>  
  </div>  
</div>
```

JSX là gì?

- ❑ Viết tắt của JavaScript XML.
- ❑ Là một phần mở rộng cú pháp cho JavaScript (do React tạo ra).
- ❑ Cho phép viết code giống HTML ngay bên trong file JavaScript.
- ❑ Nó không phải là HTML. Nó sẽ được VITE (dùng Babel) biên dịch thành các lệnh `React.createElement()` thông thường.

```
const element = <h1>Hello, world</h1>;
```

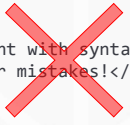


```
const element = React.createElement('h1', null, 'Hello, world');
```

Quy tắc 1: Một Component chỉ trả về 1 phần tử gốc

- ❑ Một component **phải** trả về *một và chỉ một* phần tử gốc (root element).

```
function ErrorSyntax() {  
  return (  
    <h1>This is a component with syntax errors</h1>  
    <p>Check your code for mistakes!</p>  
  );  
}
```



➡

```
<div>  
  { /* ... */ }  
</div>
```

➡

```
<React.Fragment>  
  { /* ... */ }  
</React.Fragment>
```

➡

```
<>  
  { /* ... */ }  
</>
```

Quy tắc 2: HTML attribute -> camelCase

- ❑ Vì JSX là JavaScript, một số thuộc tính HTML bị trùng tên với các từ khóa của JavaScript.

❑ class → className

❑ for → htmlFor

- ❑ Quy tắc chuyển thành camelCase

❑ onclick → onClick

❑ tab-index → tabIndex

❑ Background-color → backgroundColor

Quy tắc 3: Dùng {} để chèn JavaScript

```
function UserProfile() {  
  const name = "Tran Van A";  
  const job = "student";  
  const currentYear = 2025;  
  const birthYear = 2005;  
  
  return (  
    <>  
      <h2>Tên: {name}</h2> {/* Nhúng biến */}  
      <p>Nghề nghiệp: {job.toUpperCase()}</p> {/* Gọi hàm */}  
      <p>Tuổi: {currentYear - birthYear}</p> {/* Phép toán */}  
      <p>Ngẫu nhiên: {Math.random()}</p> {/* Gọi API của JS */}  
    </>  
  );  
}
```

Chuyển HTML tĩnh sang ReactJS

- ❑ CSS giữ nguyên
- ❑ Chỉ cần chuyển HTML sang JSX

```
<main>  
  <div class="box green">🐼</div>  
  <div class="box red">🐼</div>  
  <div class="box blue">🐼</div>  
  <div class="box yellow">🐼</div>  
  .....  
</main>
```



```
function Zodiac() {  
  return (  
    <main>  
      <div className="box green">🐼</div>  
      <div className="box red">🐼</div>  
      .....  
    </main>  
  );  
}  
export default Zodiac;
```

Nội dung

- ☐ *ReactJS – Mở đầu*
- ☐ *ReactJS – Components*
- ☐ **ReactJS – Props**
- ☐ ReactJS - TailwindCSS
- ☐ ReactJS – State

Props là gì?

- ☐ Viết tắt của **Properties**.
- ☐ Là cách để **truyền dữ liệu** từ component **Cha** xuống component **Con**.
- ☐ Giúp các component trở nên linh hoạt và có thể tái sử dụng.
- ☐ **Props** là một **object** chứa tất cả dữ liệu được truyền vào.
- ☐ **Props là Read-Only!**

Ví dụ

```
function ZodiacSign(props) {  
  return <div className={`box ${props.color}`}>{props.symbol}</div>;  
}  
export default ZodiacSign;
```



```
import ZodiacSign from "./ZodiacSign";  
function Zodiac() {  
  return (  
    <main>  
      <ZodiacSign color="green" symbol="♈" />  
      <ZodiacSign color="red" symbol="♉" />  
    </main>  
  );  
}  
export default Zodiac;
```

Props Destructuring (ES6)

```
function ZodiacSign(props) {  
  const { color, symbol } = props;  
  return <div className={`box ${color}`}>{symbol}</div>;  
}  
export default ZodiacSign;
```

```
function ZodiacSign({ color = "green", symbol = "♈" }) {  
  return <div className={`box ${color}`}>{symbol}</div>;  
}  
export default ZodiacSign;
```

Props là Array

```
import ZodiacSign from "./ZodiacSign";
function Zodiac({ listZodiacSigns = [] }) {
  const colors = ["red", "blue", "green", "purple", "brown",
"yellow"];
  return (
    <main>
      {listZodiacSigns.map((sign, index) => (
        <ZodiacSign
          key={index}
          color={colors[index % colors.length]}
          symbol={sign}
        />
      ))}
    </main>
  );
}
export default Zodiac;
```

prop.children

- ❑ **props.children** là một **prop** đặc biệt. Nó không được truyền như thuộc tính. Nó là nội dung nằm giữa thẻ đóng và thẻ mở của **component**.

```
function ZodiacSignPC({ color = "green", symbol = "♈", children }) {
  return <div className={`box ${color}`}>{children}</div>;
}
export default ZodiacSignPC;
```

```
<main>
  {listZodiacSigns.map((sign, index) => (
    <ZodiacSignPC key={index} color={colors[index % colors.length]}>
      {sign}
    </ZodiacSignPC>
  ))}
</main>
```

Truyền event qua props

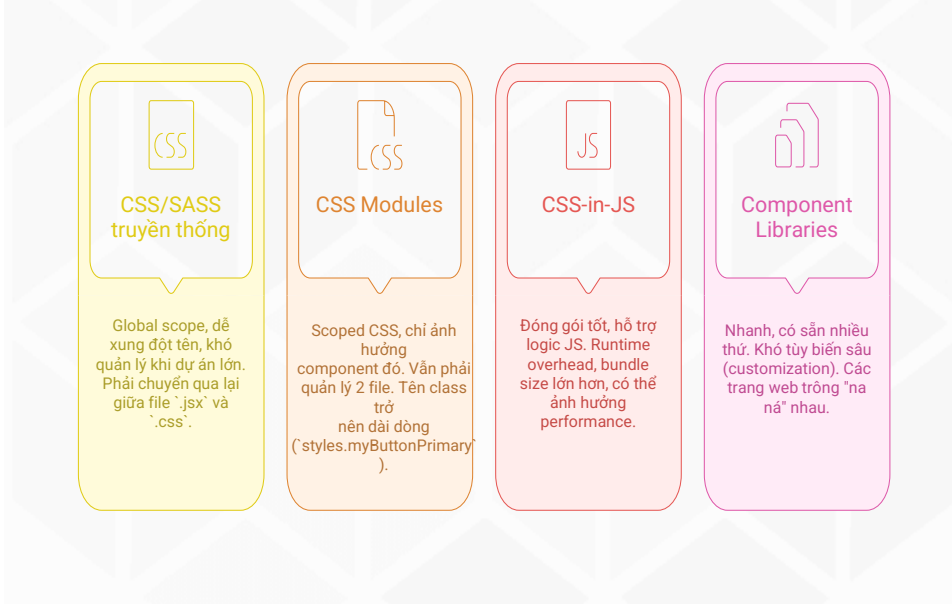
```
function ZodiacSign({ color = "green", symbol = "♈", handleMouseDown }) {  
  return (  
    <div className={`box ${color}`} onMouseDown={handleMouseDown}>  
      {symbol}  
    </div>  
  );  
}  
export default ZodiacSign;
```

```
function Zodiac({ listZodiacSigns = [] }) {  
  function handleMouseDown(e) {  
    // ...  
  }  
  return (  
    <main>  
      {listZodiacSigns.map((sign, index) => (  
        <ZodiacSign key={index} handleMouseDown={handleMouseDown}>  
          {sign}  
        </ZodiacSign>  
      ))}  
    </main>  
  );  
}
```

Nội dung

- ☐ ReactJS – Mở đầu
- ☐ ReactJS – Components
- ☐ ReactJS – Props
- ☐ **ReactJS - TailwindCSS**
- ☐ ReactJS – State

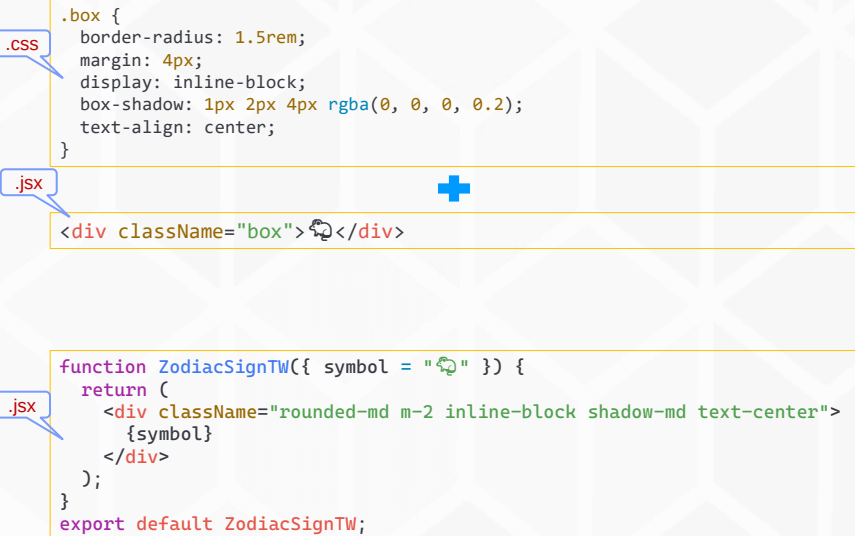
Giải pháp Styling trong React



TailwindCSS

- ❑ Là một **Utility-First CSS Framework**.
- ❑ **"Utility-First"** (Ưu tiên-Tiện ích) nghĩa là gì?
 - ❑ Nó không cung cấp các **component** làm sẵn (như `.btn`, `.card`).
 - ❑ Thay vào đó, nó cung cấp hàng ngàn các lớp CSS nguyên tử (**atomic classes**) cấp thấp.
 - ❑ Mỗi lớp làm một việc duy nhất.
- ❑ Ví dụ:
 - ❑ `p-4 -> padding: 1rem;`
 - ❑ `text-lg -> font-size: 1.125rem;`
 - ❑ `font-bold -> font-weight: 700;`

Triết lý Utility-First



React + TailwindCSS – Cặp đôi hoàn hảo

- ☐ Đồng vị trí (Co-location)
 - ☐ Đóng gói hoàn toàn trong 1 file
- ☐ Styling động (Dynamic Styling)
- ☐ Tránh xung đột tên class
- ☐ Tối ưu hiệu năng (Puging)
 - ☐ File CSS production chỉ chứa các utility thật sự cần.

Cài đặt TailwindCSS cho project

- ❑ Cài đặt: `npm install tailwindcss @tailwindcss/vite`
- ❑ Cấu hình Vite plugin (`vite.config.js`)

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react-swc'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

- ❑ Import TailwindCSS vào file CSS (`index.css`)

```
@import "tailwindcss";
```

Custom Utility

- ❑ Sử dụng arbitrary values (giá trị tùy ý)

```
background: linear-gradient(to top, #A88BEB 0%, #F1A7F1 44%, #F8CEEC 100%);
```

```
"bg-linear-to-t from-[#A88BEB] from-0% via-[#F1A7F1] via-50% to-[#F8CEEC] to-100%"
```

- ❑ Sử dụng chỉ thị `@layer` [`base/components/utilities`]

```
/* [file].css */
@layer components {
  .bg-green-gradient {
    background: linear-gradient(to top, #56B870 0%, #a5c956 100%);
  }
  .bg-red-gradient {
    @apply bg-linear-to-t from-[#C655BE] to-[#cf0404];
  }
}
```


Nội dung

- ☐ *ReactJS – Mở đầu*
- ☐ *ReactJS – Components*
- ☐ *ReactJS – Props*
- ☐ *ReactJS - TailwindCSS*
- ☐ **ReactJS – State**

Vấn đề cập nhật UI

```
function DemoState({ count }) {  
  return <div className="text-5xl font-bold">{count}</div>;  
}  
  
let count = 0;  
function App() {  
  function handleClick() {  
    count += 1;  
    console.log("Clicked", count);  
  }  
  return (  
    <div>  
      <CtnCenter>  
        <DemoState count={count} />  
        <BtnBasic onClick={handleClick}>Increase Count</BtnBasic>  
      </CtnCenter>  
    </div>  
  );  
}
```

0

Increase Count

State là gì?

- ❑ **Định nghĩa:** **State** là một đối tượng JavaScript đơn giản, chứa dữ liệu *thuộc* về component và *có thể thay đổi* theo thời gian.
- ❑ **Đặc tính cốt lõi:** Khi giá trị của **State** thay đổi, React sẽ **tự động render lại (re-render)** component đó (và các component con của nó) để phản ánh sự thay đổi ra ngoài giao diện.

useState

- ❑ Trong React hiện đại (với Function Components), chúng ta sử dụng **Hook** để thêm các tính năng của React vào component.
- ❑ Để thêm **State**, chúng ta dùng **useState Hook**.
 - ❑ Import: `import { useState } from 'react';`
 - ❑ Sử dụng: `const [stateVariable, setStateFunction] = useState(initialValue);`

Ví dụ

```
import { useState } from "react";
function App() {
  const [count, setCount] = useState(0);
  function handleClick() {
    setCount(count + 1);
    console.log("Clicked", count);
  }
  return (
    <div>
      <CtnCenter>
        <DemoState count={count} />
        <BtnBasic onClick={handleClick}>
          Increase Count
        </BtnBasic>
      </CtnCenter>
    </div>
  );
}
```

Các quy tắc sử dụng `useState`

- ☐ **KHÔNG BAO GIỜ** thay đổi State trực tiếp.
 - ☐ **Lý do:** Thay đổi trực tiếp sẽ không kích hoạt re-render. React không biết state đã thay đổi.
- ☐ Cập nhật State là Bất đồng bộ (Asynchronous).
 - ☐ React "lên lịch" cập nhật state và re-render, chứ không thực hiện ngay lập tức.
 - ☐ Cập nhật State dựa trên giá trị trước đó:

```
setCount((prevCount) => prevCount + 1);
```

State với Object và Array

❑ Lưu ý về tính **Bất biến (Immutability)**

```
const [student, setStudent] = useState({ name: "Alice", age: 20 });
function handleUpdateStudent() {
  // Wrong way:
  student.age += 1;
  setStudent(student);
  // Correct way:
  setStudent({ ...student, age: student.age + 1 });
}
```

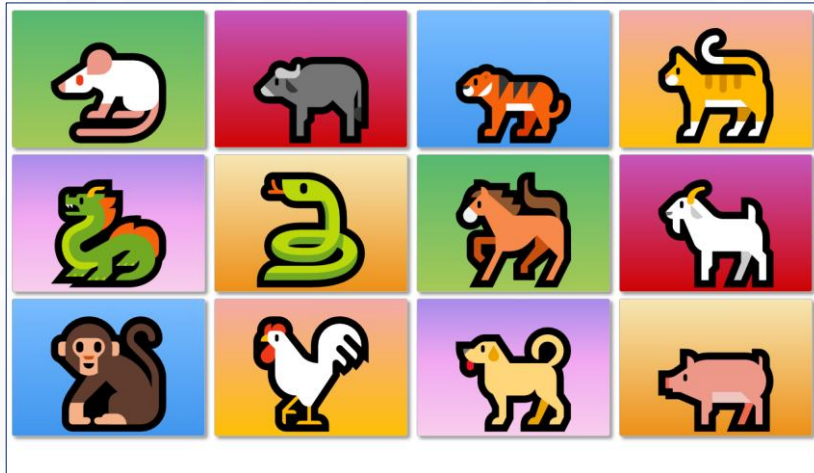
```
const [todos, setTodos] = useState([]);
function handleAddTodo() {
  // Wrong way:
  todos.push('go shopping');
  setTodos(todos);
  // Correct way:
  setTodos([...todos, 'go shopping']);
}
```

Nội dung

- ❑ *ReactJS – Mở đầu*
- ❑ *ReactJS – Components*
- ❑ *ReactJS – Props*
- ❑ *ReactJS - TailwindCSS*
- ❑ *ReactJS – State*
- ❑ **Bài tập**

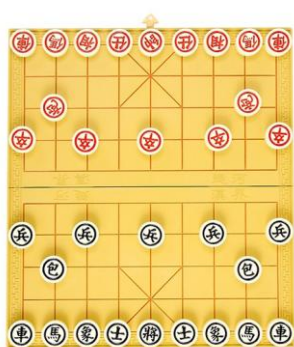
Bài tập 1 (Sử dụng ReactJS + TailwindCSS)

- ❑ Tạo trang web hiển thị như sau



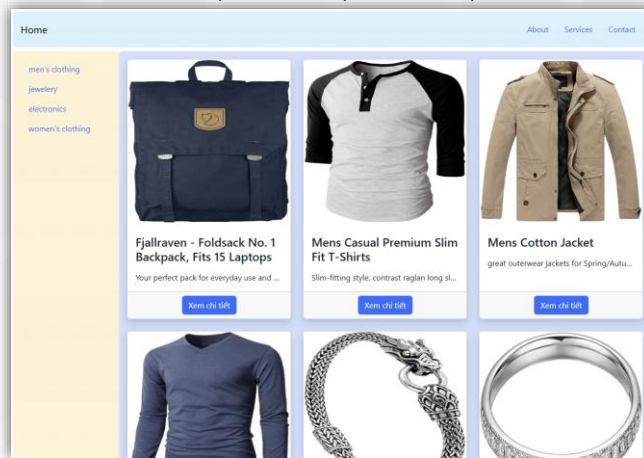
Bài tập 2 (sử dụng ReactJS + TailwindCSS)

- ❑ Dựng trang web là bàn cờ (cờ vua hoặc cờ tướng), cho phép sử dụng chuột để di chuyển các quân cờ (chỉ cho phép thay đổi vị trí vào ô đúng).



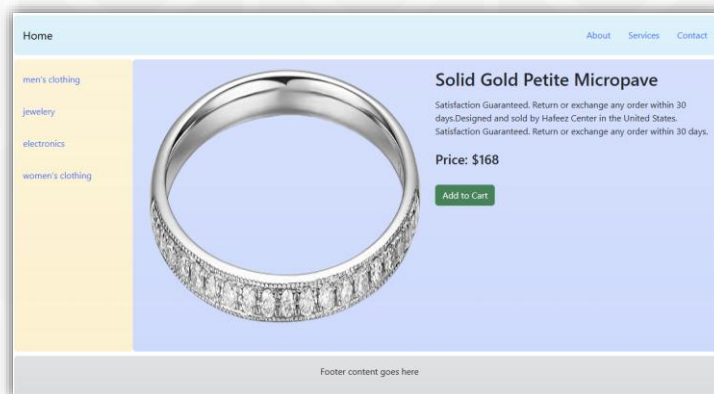
Bài tập 3 (sử dụng ReactJS + TailwindCSS)

- ❑ Với dữ liệu được cung cấp trong file “data.js” và hình ảnh trong file “imgs.zip”. Hãy tạo trang web có các chức năng như sau
 - ❑ Hiển thị danh sách tất cả sản phẩm ở Home (khi nhấn Home)



Bài tập 3 (tiếp theo)

- ❑ Khi chọn xem chi tiết thì hiển thị chi tiết sản phẩm



Bài tập 3 (tiếp theo)

- ❑ Khi chọn Danh mục thì hiển thị danh sách sản phẩm theo danh mục đó

